

Coincident Values, Distinct Schemes

Why Value Equivalence Does Not Imply Schema Equivalence

Abstract

In the context of systems design, it is common for two independently-governed classification schemes to produce identical values for the same set of entities at a given point in time. This coincidence is frequently treated as evidence that the two schemes are the same scheme, and accordingly that they may be unified, deduplicated, or brought under shared governance.

This document argues that such reasoning is a category error. Classification schemes are not defined by their current value sets. They are defined by their properties: ownership, mutability, semantic scope, and change consequence. When these properties diverge, value equivalence is coincidental and architecturally irrelevant.

Conflating schemes on the basis of coincident values imposes the wrong governance model and produces either unnecessary rigidity or unmanaged instability. The correct response is to treat the schemes as independent and to make any relationship between them explicit and governed.

This paper extends the arguments in ‘Identity, Classification, and Context Boundaries in Distributed Systems’ (hereafter: the prior paper), which established that classification and identity are distinct concerns and that classification must be treated as a first-class, explicitly governed contract across domain boundaries. The present paper addresses a further error that arises after that distinction has been accepted: the conflation of two distinct

classification schemes on the grounds that they currently produce the same values.

1. Classification Schemes as Property Sets

The prior paper established that classification answers a different question from identity. Where identity asks which exact entity is this, classification asks what kind of thing is this. Classification involves structure, hierarchy, shared semantics, and cross-context agreement.

What the prior paper did not address is the further question: when are two classification schemes the same scheme? The naive answer is: when they produce the same values. This answer is wrong.

A classification scheme is not a value set. It is a governed system with the following properties:

- Ownership: the authority responsible for defining, maintaining, and changing the scheme.
- Mutability profile: how freely values can change, under what conditions, and subject to what approval.
- Semantic scope: what question the classification answers and for which consuming systems it carries meaning.
- Change consequence: what downstream effects — including migrations, restatements, or regulatory obligations — a value change entails.

Two schemes with identical current values may diverge on every one of these properties. When they do, they are not the same scheme. They are different schemes that happen, at this moment, to agree.

2. The Coincidence Failure Pattern

The following failure pattern is common across distributed systems of any scale:

- System A owns a classification scheme for commercial or operational purposes.
- System B requires a classification scheme for reporting, compliance, or aggregation purposes.
- At the time of initial implementation, the two schemes produce identical values.
- A decision is made — often implicitly, by simply using A's values in B's context — to treat the schemes as one.
- Over time, commercial or operational requirements cause A's values to evolve freely.
- B's consumers, who assumed stability, are broken. Or alternatively: B's stability requirements constrain A's evolution, frustrating commercial agility.

The failure is not caused by either scheme behaving incorrectly. It is caused by the assumption that identical values imply identical properties. They do not.

The coincidence of values is a fact about a particular implementation at a particular moment. It says nothing about whether the schemes share an owner, a governance model, a semantic scope, or a change consequence. When any of those properties differ, the schemes must be treated as independent, regardless of whether their current values agree.

3. A Historical Precedent: Shared Configuration

The same error has recurred throughout the history of distributed systems in the domain of configuration management. It is worth examining because it is structurally identical and because its consequences are well understood.

In layered architectures built on top of shared databases, it was common for multiple applications to share a single database connection string. This string was frequently treated as a shared property, often stored once under a key such as `jdbc.url` and consumed by all applications without qualification.

The implicit assumption was: because all applications connect to the same database, they share a configuration concern. This assumption was false. What each application actually had was its own connection requirement, which happened to resolve to the same value under the current deployment topology. The correct model was not a single shared key but a set of application-specific keys — `app-a.jdbc.url`, `app-b.jdbc.url`, and so on — whose values, under a particular implementation, happened to be identical.

The consequences of failing to make this distinction became apparent whenever a database was split, a schema was partitioned, or an application needed to be migrated to a different instance. At that point, engineers discovered that the shared configuration had leaked into every integration point, leaving a thread that had to be unpicked across the entire codebase. What had appeared to be a single, simple property was in fact a set of coincident but independent requirements that had been accidentally unified.

The correct resolution — had it been applied at the outset — was straightforward: maintain independent keys, and if deduplication of the common value was desired, make it explicit through interpolation or reference. The values could still resolve identically. The distinction lay in making the independence of the requirements explicit rather than obscuring it behind a coincident value.

This is not an esoteric historical footnote. The same error, expressed in a different register, is the subject of this paper.

4. Value Equivalence as a Temporal Accident

When two classification schemes are observed to produce identical values, the correct interpretation is: at this point in time, under current conditions, these schemes agree. This is an empirical observation about the present state of two independently-evolving systems. It is not a structural or contractual claim.

Treating it as a structural claim has a predictable consequence. The two schemes, now conflated, cannot evolve independently. Any change to one is implicitly a change to the other. The governance model of whichever scheme is more tightly constrained is imposed on both.

This produces one of two outcomes, depending on which constraint dominates:

- If the less mutable scheme governs: the more mutable scheme loses the freedom its owner requires. Commercial or operational systems find themselves blocked by reporting or compliance constraints that were never intended to apply to them.

- If the more mutable scheme governs: the less mutable scheme is exposed to changes that its consuming systems cannot absorb. Reporting and compliance systems find themselves destabilised by commercial changes that carry no downstream intention.

Neither outcome is acceptable. Both are the direct consequence of treating coincident values as evidence of a shared scheme.

5. The Cost of Conflation

Beyond the immediate governance problem, conflation of classification schemes produces a set of downstream costs that compound over time.

5.1 Migration Cost

When a classification value must change in one scheme but not the other, the conflated model requires either a coordinated change affecting both — with all the migration risk and coordination cost that implies — or an undocumented divergence that is absorbed informally and inconsistently.

In the case of schemes with ledger or accounting consequences, reclassification is not a configuration change. It is a data migration event, potentially requiring restatement of historical positions. Binding a commercially-mutable scheme to one with these characteristics means commercial agility is sacrificed to accounting stability — or alternatively, that accounting stability is sacrificed to commercial agility. The former frustrates product development; the latter creates regulatory exposure.

5.2 Governance Confusion

A conflated scheme has no clear owner. If commercial and reporting teams share a single classification system, neither owns it fully. Proposed changes

become cross-team negotiations by default. The team with the strongest interest in stability will tend to resist changes; the team with the strongest interest in agility will tend to force them. This tension is not productive; it is structurally imposed by the failure to separate the schemes at design time.

5.3 Implicit Coupling

The prior paper established that in the absence of formal classification, consuming systems will infer it, and that this inference is implicit, inconsistent, brittle, and difficult to audit. The same principle applies here. When two schemes are conflated, consuming systems make implicit assumptions about which properties they are depending on. When those assumptions are violated — as they will be, when the schemes diverge — the coupling surfaces as breakage rather than as a negotiated interface change.

Explicit separation, by contrast, makes the dependency surface visible and auditable. Each consuming system depends on a specific scheme with known properties. Changes to one scheme do not implicitly affect the other.

6. Correct Design: Independent Schemes with Explicit Mapping

The resolution is straightforward in principle, though it requires deliberate design effort at the point where conflation would otherwise occur.

Where two classification schemes serve different owners, answer different questions, or carry different governance requirements, they must be modelled as independent schemes. This holds even when — especially when — their current values are identical.

Each scheme should have:

- A defined owner with clear authority over its evolution.
- A documented mutability profile, including what governance a change requires.
- A defined semantic scope: what the classification means and for which consuming systems.
- A defined change consequence: what a value change entails in terms of data migration, notification, or downstream impact.

Where a relationship exists between the two schemes — for example, where one is applied to a product configuration at the instruction of the team responsible for the other — that relationship must be made explicit. It is not a structural identity between the schemes. It is a governed act of classification applied by a defined actor under defined conditions.

The fact that two schemes currently assign the same values to the same entities is, at most, a simplification that can be exploited at the implementation level through interpolation or reference. It is not a reason to unify the schemes at the governance level. The configuration analogy is instructive: independent keys with a shared resolved value are structurally correct. A single shared key is not.

7. Relationship to Identity, Classification, and Context Boundaries

The prior paper established two foundational claims. First, that identity and classification are distinct concerns: identity addresses which entity,

classification addresses what kind. Second, that in the absence of formal classification, consumers will infer it, and that uncontrolled inference is strictly worse than governed semantics.

The present paper operates at the same level of abstraction but addresses a failure mode that arises after those claims have been accepted. It is possible to correctly separate identity from classification and still conflate two distinct classification schemes on the basis of coincident values. The error is of the same kind: treating an implementation-level coincidence as a structural or contractual fact.

The governing principle across both papers is the same: in distributed systems, the choice is not between coupling and no coupling, but between explicit, managed coupling and implicit, unmanaged coupling. Coincident values, treated as a shared scheme, produce implicit and unmanaged coupling between two independently-governed concerns. Separate, explicitly-related schemes produce coupling that is visible, auditable, and negotiable.

8. Summary

- A classification scheme is not a value set. It is a governed system with properties including ownership, mutability, semantic scope, and change consequence.
- Two schemes with identical current values may diverge on all of these properties. When they do, they are not the same scheme.
- Value equivalence is a temporal observation about the present state of two independently-evolving systems. It is not a structural or contractual claim.

- Treating coincident values as evidence of a shared scheme imposes the wrong governance model and produces either unnecessary constraint or unmanaged instability.
- The same error has recurred throughout distributed systems history in the domain of shared configuration: what appeared to be a single property was in fact a set of independent requirements that happened to resolve identically.
- The correct response is to model independent schemes as independent, make any relationship between them explicit and governed, and exploit value coincidence only at the implementation level through interpolation or reference.
- The governing principle is unchanged from the prior paper: the choice is not between coupling and no coupling, but between explicit, managed coupling and implicit, unmanaged coupling.